HUNT-SAGE · THREAT HUNTING CHEAT SHEET

KQL for Hunters

Operators, patterns and gotchas for Microsoft Sentinel and Defender Advanced Hunting.

github.com/iamjammie/hunt-sage · iamjammie.github.io/hunt-sage · Built 2026-10-31

Anatomy

```
TableName
| where Timestamp > ago(7d)                    // filter early: biggest performance win
| where Col has "value"                        // prefer 'has' over 'contains'
| extend NewCol = tolower(Col)                 // add columns
| summarize Count = count() by Key, bin(Timestamp, 1h)
| order by Count desc
| take 50                                      // = limit
```

Sentinel uses `TimeGenerated`; Defender Advanced Hunting uses `Timestamp`. Sentinel's Windows tables: `SecurityEvent`, `Event`, `SigninLogs`, `AuditLogs`, `OfficeActivity`, `CommonSecurityLog`, `DnsEvents`, `Syslog`, `AzureActivity`. Defender's: `DeviceProcessEvents`, `DeviceNetworkEvents`, `DeviceFileEvents`, `DeviceRegistryEvents`, `DeviceLogonEvents`, `DeviceImageLoadEvents`, `DeviceEvents`, `EmailEvents`, `IdentityLogonEvents`, `CloudAppEvents`.

String operators

Operator	Meaning	Notes
`==` / `=~`	equals (case-sensitive / insensitive)	Prefer `=~` for names
`has`, `has_any`, `has_all`	whole-term match, case-insensitive	**Fast** (indexed); won't match partial tokens
`contains`, `contains_cs`	substring	Slower
`startswith`, `endswith`	prefix/suffix	`endswith_cs` for case-sensitive
`in`, `in~`, `!in`	set membership	
`matches regex`	RE2 regex	`(?i)` for case-insensitive
`!has`, `!contains`	negations	

Time

```
ago(1d)    between (ago(30d) .. ago(1d))    bin(Timestamp, 15m)    startofday(now())
datetime_diff('second', t2, t1)    format_datetime(t, "yyyy-MM-dd HH:mm")
```

Aggregation toolbox

```
| summarize count(), dcount(User), make_set(Host, 20), make_list(Cmd), min(Timestamp), max(Timestamp),
            arg_max(Timestamp, *), countif(Result == "Fail"), avg(Bytes), percentile(Bytes, 95), stdev(Bytes)
            by Account, bin(Timestamp, 1h)
```

Joins

```
A | join kind=inner  (B) on Key        // matching rows
A | join kind=leftanti (B) on Key      // in A but NOT in B (new / unseen)
A | join kind=leftsemi (B) on Key      // in A with a match, B columns dropped
A | join kind=leftouter (B) on Key
```

Put the **smaller table on the right** and filter both sides before joining. Use `let` for readable subqueries and watchlists:

```
let Servers = dynamic(["srv01","srv02"]);
let Watch = _GetWatchlist('CriticalAssets') | project SearchKey;
```

Hunting patterns

Stack counting (rare values)

```
DeviceProcessEvents
| where Timestamp > ago(30d)
| summarize Hosts = dcount(DeviceName), Runs = count() by FolderPath, FileName, SHA256
| where Hosts <= 2
| order by Runs desc
```

First-seen (new vs. baseline)

```
let base = DeviceNetworkEvents | where Timestamp between (ago(30d) .. ago(1d)) | distinct RemoteUrl;
DeviceNetworkEvents | where Timestamp > ago(1d) | where RemoteUrl !in (base) | summarize by RemoteUrl, DeviceName
```

Sequence: A then B within 5 minutes on the same host

```
let A = DeviceProcessEvents | where FileName =~ "winword.exe" | project DeviceName, AT = Timestamp, AProc = ProcessI
d;
let B = DeviceProcessEvents | where FileName in~ ("cmd.exe","powershell.exe") | project DeviceName, BT = Timestamp,
Cmd = ProcessCommandLine, ParentPid = InitiatingProcessId;
A | join kind=inner B on DeviceName
| where BT between (AT .. AT + 5m) and ParentPid == AProc
```

Time-series anomaly

```
let ts = SigninLogs | where TimeGenerated > ago(14d) | make-series Fails = countif(ResultType != 0) default=0 on Tim
eGenerated step 1h by UserPrincipalName;
ts | extend (Flag, Score, Baseline) = series_decompose_anomalies(Fails, 2.5)
   | mv-expand TimeGenerated to typeof(datetime), Fails to typeof(long), Flag to typeof(int)
   | where Flag == 1
```

Parse JSON / extract

```
| extend P = parse_json(AdditionalFields) | extend Task = tostring(P.TaskName)
| extend Url = extract(@"https?://([^/\s]+)", 1, ProcessCommandLine)
| parse ProcessCommandLine with * "-File " Script " " *
```

Base64 decode

```
| extend Decoded = base64_decode_tostring(Blob)       // UTF-8 only: PowerShell -enc is UTF-16LE; decode in CyberChe
f
```

Entropy-like proxy: count distinct characters

```
| extend Label = tostring(split(Name, ".")[0]) | extend Len = strlen(Label)
```

Gotchas

- `has` needs a delimiter-bounded term: `has "mimikatz"` matches `mimikatz.exe` but not `xmimikatzx`.
- Backslashes in strings: use `@"C:\Windows\..."` (verbatim) or double them.
- `SecurityEvent` text columns such as `LogonType` are numeric; `TicketEncryptionType` is a **string** like `"0x17"`.
- `summarize` loses columns not in `by` /aggregations; use `arg_max(t, *)` to keep a row.
- Advanced Hunting retention is ~30 days; for older data use Sentinel/Log Analytics.
- Test performance with `| take 10` and narrow `ago()` first, then widen.